ISIS-2111 PLE

Proyecto 1

Definición de la Gramática de Verilang

Amelia Serrano Arango – 202221556
Cristian David Ortiz Sanchez – 202311404

Febrero 2026

1. Alfabeto (Σ)

1.1 Palabras Reservadas

```
"defmodule" "using" "defspace" "defoperator" "defexpression" "defrule" "defvar"
"end" "forall" "exists" "in" "defer"
```

1.2 Símbolos y Operadores

```
( ) [ ] : . ,
+ - * / ** % < > <= >= <> = ->  =>
and or neg in
"a" "b" "c" "d" "e" "f" "g" "h" "i" "j" "k" "l" "m"
"n" "o" "p" "q" "r" "s" "t" "u" "v" "w" "x" "y" "z"
"A" "B" "C" "D" "E" "F" "G" "H" "I" "J" "K" "L" "M"
"N" "O" "P" "Q" "R" "S" "T" "U" "V" "W" "X" "Y" "Z"
"0" "1" "2" "3" "4" "5" "6" "7" "8" "9"
```

2. Conjunto de No Terminales ($\mathcal{N}$)

```
N = { Module, Using, SpaceDef, OperatorDef, VarDef, RuleDef, ExpressionDef, Type,
LogicalExpression, Atom, OrExpr, AndExpr, EqExpr, AddExpr, MultExpr, PowExpr,
   UnaryExpr, Application,
AttributeList, Attribute, Letter, LowerLetter, UpperLetter, Number, Identifier,
IntLiteral, FloatLiteral, CharLiteral }
```

3. Símbolo Inicial (S_0)

$$S_0 = Module$$

4. Reglas de Producción ($\mathcal{P}$)

4.1 Estructura del Módulo

```
Module ::= "defmodule" Identifier
          { Using | SpaceDef | OperatorDef | VarDef | RuleDef | ExpressionDef }
          "end"

Using ::= "using" Identifier
```

4.2 Definiciones

```
SpaceDef ::= "defspace" Identifier [ "<" Identifier ] "end"
OperatorDef ::= "defoperator" Identifier ":" Identifier { "->" Identifier }
    [ AttributeList ] "end"
VarDef ::= "defvar" Identifier ":" Type { "," Identifier ":" Type } "end"
RuleDef ::= "defrule" Application "->" Application "end"
ExpressionDef ::= "defexpression" LogicalExpression [ AttributeList ] "end"
```

4.3 Expresiones

```
LogicalExpression ::= OrExpr
OrExpr ::= AndExpr { "or" AndExpr }
AndExpr ::= EqExpr { "and" EqExpr }
EqExpr ::= AddExpr { ( "=" | "<>" | "<" | ">" | "<=" | ">=" | "" | "=>" | "in" )
    AddExpr }
AddExpr ::= MultExpr { ( "+" | "-" ) MultExpr }
MultExpr ::= PowExpr { ( "*" | "/" | "%" ) PowExpr }
PowExpr ::= UnaryExpr { "**" UnaryExpr }
UnaryExpr ::= [ "neg" ] Atom | ( "forall" | "exists" ) Identifier "in"
    Identifier "." LogicalExpression
Atom ::= Identifier | IntLiteral | FloatLiteral | CharLiteral | Application | "("
     LogicalExpression ")"
Application ::= "(" Identifier { LogicalExpression } ")"
```

4.4 Elementos

```
Type ::= Identifier
AttributeList ::= "[" Attribute { Attribute } "]"
Attribute ::= Identifier [ ":" Identifier ]
Identifier ::= Letter { Letter | Number | "-" }
Letter ::= LowerLetter | UpperLetter
LowerLetter ::= "a" | "b" | "c" | "d" | "e" | "f" | "g" | "h" | "i" | "j" | "k" |
    "l" | "m" | "n" | "o" | "p" | "q" | "r" | "s" | "t" | "u" | "v" | "w" | "x" |
    "y" | "z"
UpperLetter ::= "A" | "B" | "C" | "D" | "E" | "F" | "G" | "H" | "I" | "J" | "K" |
    "L" | "M" | "N" | "O" | "P" | "Q" | "R" | "S" | "T" | "U" | "V" | "W" | "X" |
    "Y" | "Z"
Number ::= "0" | "1" | "2" | "3" | "4" | "5" | "6" | "7" | "8" | "9"

IntLiteral ::= Number { Number }
FloatLiteral ::= Number { Number } "." Number { Number }
CharLiteral ::= "'" Letter "'"
```